

Explication du notebook du modèle HMM et de son extension FSHMM

I. MODELE HMM

Bloc 1 : Chargement, préparation et normalisation des données pour l'entraînement

 Ce bloc marque le début de la **Phase 1 : Entraînement initial du modèle HMM**. Il comprend plusieurs étapes essentielles :

1. **Importation des bibliothèques** nécessaires :
 - GaussianHMM pour le modèle de Markov caché qui se trouve dans le script python *hmm.py*,
 - StandardScaler pour la normalisation des données,
 - pandas et numpy pour le traitement de données.
2. **Chargement de la base de données** : fichier Excel contenant les rendements journaliers des portefeuilles factoriels construits.
3. **Nettoyage et structuration des données** :
 - Conversion de la colonne Date en index temporel (si applicable),
 - Suppression de la colonne Date une fois l'index défini.
4. **Définition de la période d'entraînement initial** :
 - Le modèle est entraîné sur les 1852 premières observations (environ 5 ans de données).
5. **Standardisation** des variables (facteurs) pour garantir que chacun ait une moyenne nulle et une variance unitaire, ce qui est essentiel pour la stabilité du modèle HMM.

PHASE 1 : Entraînement initial du HMM

```
# Import des biblio
from hmmlearn.hmm import GaussianHMM
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd

# === 1. Charger ta base ===
df = pd.read_excel("C:/Users/Lenovo i7/OneDrive - ENSEA/Stage CGF Gestion/Documents/Mémoire/Data/EDOUARD/Rendement/rendements_portefeilles_corr (1).xlsx", engine='openpyxl')

# === 2. Préparer les données ===
# S'assurer que les dates sont bien dans l'index (si tu as une colonne Date)
df.index = pd.to_datetime(df['Date']) # si applicable
df = df.drop(columns=['Date']) # si applicable

# Étape 1 : Sélection de la période d'entraînement
train_end = 1852
df_train = df.iloc[:train_end].copy()

# Étape 2 : Standardisation
scaler = StandardScaler()
X_train = scaler.fit_transform(df_train)
```

Bloc 2 — Entraînement du modèle HMM

 Ce bloc effectue l'entraînement proprement dit du **modèle de Markov caché (HMM)**, à partir des données standardisées sur la période d'entraînement (1852 observations).

Voici les paramètres utilisés :

- `n_components=2` : le modèle identifie **2 régimes de marché cachés** (états).
- `covariance_type='full'` : chaque régime possède une matrice de covariance complète.
- `n_iter=1000` : le nombre maximal d'itérations de l'algorithme EM pour convergence.
- `random_state=42` : permet de reproduire les résultats en fixant l'aléa

```
# Étape 3 : Entraînement du modèle HMM (2 états cachés)
hmm_model = GaussianHMM(n_components=2, covariance_type='full', n_iter=1000, random_state=42)
hmm_model.fit(X_train)
```

NB : L'on peut changer à volonté ces hyperparamètres du modèle. Dans la littérature, il est conseillé d'entraîner plusieurs modèles HMM avec *deux, trois, quatre, cinq* ou *six états* (régimes) cachés afin de voir celui qui s'adapte le mieux à sa situation et le choisir.

Bloc 3 — Prédiction et visualisation des régimes cachés

 Une fois le modèle HMM entraîné, ce bloc :

◆ Étape 4 : Prédiction des régimes cachés

Le modèle prédit, pour chaque jour de la période d'entraînement, **l'état latent (régime de marché)** dans lequel se trouve le système (0 ou 1, car on a deux régimes).

◆ Étape 5 : Stockage des régimes

On crée un DataFrame `df_regimes_hmm` qui associe chaque date à son **régime latent estimé** (*Regime_brut*). On peut aussi afficher le nombre de jours passés dans chaque régime.

◆ Étape 6 : Visualisation du régime détecté dans le temps

Ce bloc produit un **graphe temporel en escalier** permettant de visualiser l'évolution des régimes de marché prédits jour après jour.

```
# Étape 4 : Prédiction des états cachés
hidden_states = hmm_model.predict(X_train)

# Étape 5 : Stockage des régimes
df_regimes_hmm = pd.DataFrame({
    'Date': df_train.index,
    'Regime_brut': hidden_states
}).set_index(['Date'])

df_regimes_hmm['Regime_brut'].value_counts()

Regime_brut
0    1514
1     338
Name: count, dtype: int64

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Suppose que tu as déjà obtenu ceci :
# df_regimes_hmm = DataFrame avec colonnes ['Regime_brut'] et Date en index

plt.figure(figsize=(14, 3))
plt.plot(df_regimes_hmm.index, df_regimes_hmm['Regime_brut'], drawstyle='steps-post')
plt.title("")
plt.xlabel("Date")
plt.ylabel("Régime caché")
plt.yticks([0, 1])
plt.grid(True)
plt.tight_layout()
plt.show()
```

📌 Bloc 4 — Réentraînement jour par jour et confirmation des régimes

◆ Étape 1 : Définition des paramètres

On définit ici les bornes de la **période de réentraînement** et le **paramètre de confirmation** `d`, qui permet de **filtrer les changements de régime trop volatils**.

◆ Étape 2 : Réentraînement jour par jour

À chaque itération `t`, on réentraîne un nouveau modèle HMM sur toutes les données disponibles jusqu'au jour `t`, puis on prédit **le régime du jour `t`**.

◆ Étape 3 : Application de la règle de confirmation

Le changement de régime est **confirmé uniquement** s'il reste identique pendant au moins d jours consécutifs.

```
PHASE 2 : Ré-entraînement jour par jour

# Import des biblio
from hmelearn.hmm import GaussianHMM
from sklearn.preprocessing import StandardScaler

# Paramètres
start_retrain = 1853
end_retrain = 2107
start_idx = 1853          # début du réentraînement (02/01/2023)
end_idx = 2107
d = 5 # Nombre de jours de confirmation

# Initialisation
regimes_bruts = []
dates_retrain = df.index[start_retrain:end_retrain]
scaler = StandardScaler()

# Réentraînement jour par jour
for t in range(start_retrain, end_retrain):
    df_window = df.iloc[:t]
    X_window = scaler.fit_transform(df_window.select_dtypes(include=[np.number]))

    model = GaussianHMM(n_components=2, covariance_type='full', n_iter=1000, random_state=42)
    model.fit(X_window)
    z = model.predict(X_window)[-1] # Dernier état estimé
    regimes_bruts.append(z)

# Construction du DataFrame des régimes bruts
df_regimes_hmm_rt = pd.DataFrame({
    'date': dates_retrain,
    'Regime_brut': regimes_bruts
}).set_index('date')

# Application de la règle de confirmation (d=5)
regimes_confirmez = []
dernier_regime_confirme = regimes_bruts[0]

for i in range(len(regimes_bruts)):
    if i < d:
        regimes_confirmez.append(dernier_regime_confirme)
        continue
    derniere_fenetre = regimes_bruts[i-d+1:i+1]
    if all(r == derniere_fenetre[0] for r in derniere_fenetre):
        dernier_regime_confirme = derniere_fenetre[0]
        regimes_confirmez.append(dernier_regime_confirme)

# Ajout des régimes confirmés
df_regimes_hmm_rt['Regime_confirme'] = regimes_confirmez

df_regimes_hmm_rt['Regime_confirme'].value_counts()
```

📌 Bloc 5 — Détection des points de réallocation mensuelle

Ce bloc identifie les **moments où un changement de régime confirmé justifie une nouvelle allocation du portefeuille**.

L'idée est de ne réallouer **qu'au début du mois** (tous les 22 jours environ), **et seulement si** un changement de régime est détecté sur le mois précédent.

◆ Étape 1 : Définition des bornes de test et de l'espacement

◆ Étape 2 : Construction des index et dates de début de mois

◆ Étape 3 : Règle de déclenchement

Pour chaque mois, on regarde s'il y a eu **au moins un changement de régime confirmé** sur les 22 jours précédents.

◆ Étape 4 : Intégration dans le DataFrame

On ajoute une colonne *Declencher_reallocation* à *df_regimes_hmm_rt*, qui prend la valeur *True* uniquement si une réallocation doit être effectuée à cette date.

```
PHASE 3 : Construction des portefeuilles factorielles dynamiques

Règle de déclenchement

# Début et fin de la phase de test + réentraînement
start_test = 1853
end_test = 2107 # jusqu'à 2106 inclus
step = 22 # début de mois = tous les 22 jours

# Index relatifs dans df_facteurs
debut_mois_idx = list(range(start_test, end_test, step))

# Dates correspondantes
dates_debut_mois = df.index[debut_mois_idx]

# Règle de déclenchement : vérifier changement de régime sur les 22 jours précédents
reallocation_flags = []

for i, t in enumerate(debut_mois_idx):
    if i == 0:
        reallocation_flags.append(False)
        continue

    # Plage dans df_regimes_hmm_rt : relative à t - start_test
    debut = debut_mois_idx[i - 1] - start_test
    fin = debut_mois_idx[i] - start_test

    fenetre_regimes = df_regimes_hmm_rt.iloc[debut:fin]['Regime_confirme']
    changement = fenetre_regimes.diff().fillna(0) != 0
    reallocation_flags.append(changement.any())

# Initialisation de la colonne "Declencher_reallocation"
df_regimes_hmm_rt['Declencher_reallocation'] = False

for date, flag in zip(dates_debut_mois, reallocation_flags):
    if date in df_regimes_hmm_rt.index:
        df_regimes_hmm_rt.loc[date, 'Declencher_reallocation'] = flag
```

NB : Dans le notebook, après ce code permettant d'avoir la règle de déclenchement, nous avons ensuite les différentes fonctions et bloc de codes qui permettent de construire les différents portefeuilles.

Nous sautons ces étapes dans ce fichier qui sont répétitives et allons directement au niveau de la mesure de performances.

📌 Bloc 6 : Mesure de performance des portefeuilles construits

◆ Étape 1 : Regrouper les portefeuilles dynamiques et benchmarks dans un seul DataFrame

Cette fonction *performance_stats()* calcule plusieurs **indicateurs clés de performance** à partir des valeurs cumulées d'un portefeuille :

- *rendement_ann* : rendement annualisé

- *volatilite_ann* : volatilité annualisée
- *downside_risk_ann* : risque baissier annualisé (utilisé dans le ratio de Sortino)
- *IR* : ratio de Sharpe
- *Sortino* : ratio de Sortino (plus robuste au risque baissier)
- *Skew et Kurt* : asymétrie (skewness) et kurtosis des rendements
- *DD* : drawdown maximum
- *DD days* : nombre de jours au drawdown maximum

Ces mesures sont ensuite retournées sous forme de *pd.Series*.

◆ Étape 2 : Construction des index et dates de début de mois

◆ Étape 3 : Application de la fonction à tous les portefeuilles

- On **applique la fonction `performance_stats`** à chacun des portefeuilles.
- Les résultats sont concaténés verticalement (`axis=1`) puis transposés (`.T`) pour que chaque portefeuille devienne une ligne et chaque indicateur une colonne.
- Cela forme une **matrice de performances finales** : *tableau_resultats*.

◆ Étape 3 : Affichage propre du tableau

- Tu ajustes le format d’affichage pour que tous les chiffres soient montrés avec 4 décimales (`'%.4f'`).
- Enfin, tu affiches proprement le tableau de résultats.

II. MODELE FSHMM

La seule différence entre le modèle FSHMM et le modèle HMM se trouve au niveau de l’entraînement du modèle. Les autres étapes restent quasiment les mêmes.

Nous expliquons donc uniquement cette étape ici. Pour les autres étapes du notebook *modèle FSHMM*, se référer aux explications apportées pour le modèle FSHMM.

Explication ligne par ligne :

« *model = GaussianFSHMM(...)* »

Cette ligne crée une instance du modèle **FSHMM (Feature Saliency Hidden Markov Model)**. Ce modèle est une extension du HMM classique, capable **d’identifier automatiquement les variables pertinentes (feature selection)**.

La classe *GaussianFSHMM()* se trouve dans les scripts python *hmm.py* et *base.py*.

Paramètres :

- *n_components=2*
- Tu spécifies **2 états cachés**, donc ton modèle va détecter deux **régimes de marché** (ex. régime haussier / baissier).
- *k=50*
C'est le **paramètre de pénalité L1** pour la saillance des variables (feature saliency). Plus k est élevé, plus le modèle **force la sélection d'un petit nombre de variables** pertinentes.
- *algorithm='viterbi'*
L'algorithme utilisé pour **trouver la séquence la plus probable d'états cachés** est le Viterbi.
Ce n'est pas l'algorithme EM d'estimation, mais celui de détection ex-post des états.
- *covariance_type='diag'*
Suppose que **les covariances entre les variables sont nulles**. Seules les variances sont apprises.
C'est utile si l'on veut éviter d'estimer des matrices de covariance complètes (lourdes à estimer en grande dimension).
- *init_params='stmc'*
Ce paramètre dit au modèle ce qu'il doit **initialiser aléatoirement** avant de commencer l'algorithme EM :
 - 's': les probabilités initiales (startprob_)
 - 't': la matrice de transition (transmat_)
 - 'm': les moyennes (means_)
 - 'c': les covariances (covars_)
- *n_iter=1000*
→ Le nombre maximal d'itérations de l'algorithme EM pour converger.
- *random_state=42*
Pour garantir la reproductibilité des résultats (fixe les tirages aléatoires).